Design patterns and backend architecture

Chapter 9: Final one-page recall

Practice designs, not claims about an employer.

9. Final one-page recall

- **Name the problem before the pattern.** A pattern is a tool, not the goal.
- **Principles:** cohesive responsibility; reliable contracts; business policy independent from infrastructure; protect invariants.
- **Simple object patterns:** Strategy varies a rule; Factory selects creation; Builder assembles a complex value; Adapter translates; Facade simplifies; State protects lifecycle transitions.
- **Architecture:** layers separate roles; hexagonal protects the core; modular monolith keeps modules in one deployment; microservices add independent ownership and distributed costs.
- **Data:** local transactions protect one boundary; outbox connects a committed change to an event; saga coordinates local steps; CQRS separates read and write models.
- **Failures:** deadlines, retry safety, circuit breakers, bulkheads, idempotency, reconciliation, and monitoring must fit together.
- **Professional answer:** problem -> choice -> implementation -> tradeoff -> failure handling -> evidence.
- **Honesty:** "I implemented __; my team used __; I understand __ conceptually." These are different claims.

Two questions for your interviewer: "How does the team decide service boundaries and data ownership?" and "Which failure mode has been most challenging in your payment or servicing workflows?"

Reference scope: Official documentation linked near the relevant architecture patterns was consulted on October 22, 2026. These patterns are general practice, not a claim about an American Express internal design. The interview examples are tailored to the experience described in the four resume versions you supplied.